

AlpacaAgent

The operator's handbook

A practical, detailed guide to setup, research, shadow testing, controlled paper operation, and daily review.

DETAILED WINDOWS EDITION

Based on the local implementation, with corrected commands, practical examples, and explicit operational boundaries.

Prepared 18 September 2026

Source revision 235b087

Companion volumes: handbook, use cases, and desk reference

Contents

Choose a chapter below. PDF bookmarks and contents links navigate directly to each chapter.

1. The most effective way to use the system	3
2. Understand the roles and boundaries	4
3. Set up a clean Windows session	5
4. Open the dashboard in the right mode	6
5. Research before enabling anything	7
6. Run a genuine shadow-testing phase	8
7. Know exactly what can become an order	9
8. Conduct a controlled paper pilot	10
9. Understand how exits really work	11
10. Build an efficient daily and weekly routine	12
11. Limits, evidence, and the next learning cycle	13

1. The most effective way to use the system

AlpacaAgent works best as an evidence-gathering paper-trading laboratory. Use it to learn whether a repeatable setup survives data checks, risk rules, order handling, and actual paper fills. The immediate objective is a process you can explain and reproduce: what the system saw, why it acted or refused, what happened at the broker, and what you learned.

This guide describes the local AlpacaAgent repository, not every feature offered by the Alpaca brokerage platform. It was checked against source revision 235b087 on September 18, 2026. Local documentation contains older milestones and some contradictory examples; the implementation takes precedence here. No account was queried and no trading controls were changed while preparing these guides.

Your recommended progression

1. Learn the dashboard and read existing reports without making orders possible.
2. Reproduce underlying-price backtests from saved bars. Write a research hypothesis before looking for the best result.
3. Run broker diagnostics and standalone shadow scans. Record missing data and failures separately from valid scans with no ideas.
4. Review one executable playbook and a narrow symbol universe. Approve only after you can explain its evidence and limitations.
5. Run a supervised paper pilot with unchanged limits. Review order outcomes and exit behavior, not just profit.
6. Expand only after resolving operational defects and evaluating enough observations. There is no live-trading switch in this project.

Start with the fewest moving parts that answer your question. A historical-research session needs neither a running controller nor an armed control file. A dashboard-reading session does not need broker credentials.

How to read this guide set

Volume 1 explains the operating workflow in order. Volume 2 supplies worked use cases, agent prompts, and research templates. Volume 3 is the desk reference for commands, failures, recovery, and the meaning of records. Read Volume 1 first; keep Volume 3 nearby when running the system.

What a successful first week looks like

You can open the dashboard, identify the timestamp of its evidence, reproduce a report, distinguish a scanner failure from no signal, explain every readiness gate, and locate the corresponding cycle or journal record. More trades are not a success criterion. A refusal that protects an invariant is useful evidence.

Do not treat the historical account and subscription descriptions in `HANDOVER.md` as a current account check. They are notes from an earlier session. The same applies to references to previously exposed credentials: if those credentials have not already been rotated, rotate them before reuse.

2. Understand the roles and boundaries

Star: idea generation

Star's scanner combines completed daily bars, deterministic signals, and eligible option contracts. It can produce shadow ideas, enabled proposals, and reasons for skipping candidates. It does not own the broker connection and does not decide how much risk the account may take.

The normal CLI universe is SPY, QQQ, and IWM. DIA is also accepted through --symbols. These are the supported ETF inputs for this workflow; the fact that a chat tool can describe another stock does not add that stock to the scanner.

Moon: deterministic rules

Moon evaluates a fully specified idea against a fresh, reconciled risk state. It recomputes important quantities rather than accepting a persuasive thesis. A chat conversation with Moon explains rules; it does not modify the deterministic engine or make an exception.

Houston: execution and reconciliation

Houston's executor compares broker information with the local activity history, fill ledger, and order journal. Orders pass through a durable reserve, claim, and submit sequence. The broker transport makes one submission attempt per claimed authorization. An uncertain outcome remains an incident for reconciliation, not permission to retry.

Astra and Nova: interpretation

Astra explains workspace status and saved records. Nova is available in generative mode as a general assistant. The agent names are different conversational roles, not five independent trading accounts or five risk budgets. All trading limits are shared.

The controller coordinates the work

One controller cycle reconciles first, resolves eligible terminal intents, checks exits, then considers entries. If reconciliation fails, the cycle does not continue into ordinary trading. The runtime lock prevents two controllers from owning the same runtime directory.

The dashboard is a view of records. Opening it does not start the controller, and opening a served workspace without --every does not start periodic trading cycles. The machine, terminal process, credentials, network, and applicable data feeds must be available for scheduled operation.

Capabilities you should not assume

The current executor supports long single-leg calls and puts. Some debit spreads can be constructed and validated earlier in the pipeline, but multi-leg execution is refused. Chat cannot place or cancel orders, arm controls, approve playbooks, release intents, or edit limits. There is no cancel/replace workflow, automatic order chasing, complete assignment/exercise handling, or options-level historical P&L backtest.

3. Set up a clean Windows session

Establish the working directory and Python

Run the examples from the repository root in PowerShell. Python 3.11 or newer is required. In this documentation-building environment, a plain python command was not on PATH; your normal terminal may differ. Check before diagnosing the application itself.

```
Set-Location 'C:\Users\eveli\OneDrive\Desktop\AlpacaAgent'  
python --version  
python -m pip install -e .
```

If python is not recognized, try py -3 --version. Use py -3 in place of python throughout if that is your working interpreter, or use an installed Python executable's absolute path with PowerShell's & operator. Install into the same interpreter you will use to launch the app.

The editable installation allows python -m alpaca_agents... to find the package. Runtime code uses the Python standard library. Node is needed for the browser-side test suite, not normal trading operation.

Keep credentials out of documents and chat

The executor reads ALPACA_PAPER_API_KEY and ALPACA_PAPER_API_SECRET. Market data reads MASSIVE_API_KEY or POLYGON_API_KEY. Optional generative chat reads ANTHROPIC_API_KEY. A .env file is not automatically loaded by this application.

If you already maintain a protected local key-loading script, load it into the terminal you will use. The path below is the project's documented convention; it is not proof the file exists or is current.

```
. 'C:\Users\eveli\keys.ps1'
```

Do not print the environment, paste keys into agent chat, include them in screenshots, or save them in this repository. A process receives its environment at launch; after updating credentials, restart the affected process. Prefer separate shells with only the credentials needed for research, broker diagnostics, or combined controller operation.

Initialize controls only for a new setup

The following creates the directory and writes DISABLED. Do not blindly overwrite an existing operator-selected mode, especially if positions are held: DISABLED also prevents automatic exits.

```
python -c "from pathlib import Path; Path('runtime').mkdir(exist_ok=True)"  
@'  
from pathlib import Path  
Path('runtime/trading-control').write_text('DISABLED', encoding='utf-8')  
'@ | python
```

Python writes predictable UTF-8 marker files. Avoid relying on PowerShell redirection, whose encoding differs by version. The exact trimmed marker text matters.

Margin-paper acknowledgement

If reconciliation identifies a margin paper account, the current implementation supports an explicit acknowledgement marker. It still enforces its local cash semantics and \$2,000 capital cap. This does not convert the brokerage account into a cash account or waive broker rules. Only create the marker after understanding that distinction.

```
@'  
from pathlib import Path  
Path('runtime/paper-margin-acknowledged').write_text('ACKNOWLEDGED', encoding='utf-8')  
'@ | python
```

4. Open the dashboard in the right mode

Static mode: saved evidence, no server

Double-click Open-Dashboard.cmd, or generate and open the dashboard manually:

```
python -m alpaca_agents.dashboard  
Start-Process .\runtime\dashboard.html
```

The launcher rebuilds the HTML from local records. It does not reconcile with Alpaca, scan markets, or submit orders. Static chat is a keyword-based guide over the embedded snapshot. Refreshing an old HTML file does not create a new broker check; rebuild the file after records change.

Served mode: fresh local reads per question

```
python -m alpaca_agents.controller --runtime runtime --serve
```

Open the localhost URL printed in the terminal and keep that terminal open. The port is selected at launch. This mode reads current local records when you ask a question; those records may still describe old broker observations. No periodic cycles run unless --every is supplied.

Select Houston and send the exact message reconcile to request a dry controller diagnostic. That request can contact the broker, import activity, resolve eligible terminal records, and write diagnostics, but it cannot submit orders or reserve new entries through the chat path. It needs the appropriate process credentials. Its ability to scan also depends on control mode and market-data access.

Scheduled served mode

```
python -m alpaca_agents.controller --runtime runtime --serve --every 300
```

This adds scheduled cycles without order submission. It owns the same runtime lock as a terminal-only controller. Do not launch another controller on that runtime. A served schedule can pause at market close or on an incident while the browser server remains available. Restart the session deliberately after checking the cause; do not assume it resumes tomorrow.

Read the dashboard in this order

1. Overview: control mode and recorded state. Check data-quality warnings before trusting numbers.
2. Activity: latest cycle timestamp and reconciliation reasons. Recent page activity is not a recent broker check.
3. Positions: remaining ledger inventory and basis. Basis is not a current mark-to-market portfolio value.
4. Outstanding intents and risk checks: unresolved work, loss latch, and entry blockers.
5. Scanner: source freshness, errors, shadow ideas, and skips.
6. Research: report dates, samples, mean and median R, and downside observations.
7. Chat: ask for an explanation of specific records you have already identified.

The served browser polls metadata about every 30 seconds. That updates connection/age indicators, not every main panel. Refresh the page to rebuild the displayed panels from current local records. Unknown means unavailable evidence; it is not the same as an empty account or zero loss.

5. Research before enabling anything

Ask a testable question

A useful question is: "Does this unchanged trend definition show reasonably consistent underlying-price outcomes across the chosen period and symbols?" A weak question is: "Which combination produces the biggest number?" Write down the universe, dates, playbook, and acceptance criteria before comparing results.

The backtest names are trend, oversold_bounce, and breakout. Controller playbook names are different: trend_directional, trend_debit_spread, oversold_bounce, and breakout_continuation. Research on trend does not establish that an executable option contract will satisfy the premium and liquidity limits.

Fetch once and save a reproducible cache

Both --start and --end are required for a network history run. These dates are an example research window ending at the previous completed session for this guide's date; choose your own completed period.

```
python -m alpaca_agents.backtest --symbol QQQ `
--start 2023-01-01 --end 2026-09-17 `
--playbooks trend oversold_bounce breakout `
--save-bars runtime/bars-QQQ.json `
--output runtime/bt-QQQ.json
```

PowerShell's backtick must be the final character on its line. A single-line command is equivalent. Repeat for other supported ETFs when the research question calls for them.

Repeat from the same data

```
python -m alpaca_agents.backtest --symbol QQQ `
--bars-file runtime/bars-QQQ.json `
--playbooks trend oversold_bounce breakout `
--output runtime/bt-QQQ.json
```

This run needs no market-data request. Keep the source bars and report together. Use bt-*.json output filenames for dashboard discovery; the CLI's default backtest-QQQ.json does not match that pattern. Generative chat's symbol lookup specifically uses bt-QQQ.json, so retain a canonical report per symbol. Archive experiment variants separately to avoid accidental duplicate comparison.

Read the full result, not only expectancy

Begin with resolved count and exclusions. Then read mean R, median R, profit factor, worst loss R, win rate, holding time, and exit reasons together. Fewer than 30 resolved observations fails the code's minimum-sample flag. Thirty is a minimum software threshold, not statistical proof.

An R-multiple measures the underlying move relative to the modeled stop distance. For an illustrative long entry at 100, stop at 98, and exit at 103, the underlying return is +1.5R. That does not mean an option made \$150, 150%, or any predictable premium return. Option prices, IV, theta, bid-ask costs, and fees are not modeled.

Read the caveats and trades array as well as summary. A profitable stop-rule exit is possible: outcome is the sign of R; exit reason describes which condition ended the trade. no_fill, no_trade, and unresolved are different categories, not ordinary losses.

6. Run a genuine shadow-testing phase

Separate connectivity testing from signal testing

With trading-control set to DISABLED, the controller reconciles but skips the entry/scanner stage. Older operator notes describe a disabled loop as if it scanned everything. To study ideas while keeping the control file disabled, use the standalone shadow scanner separately.

First check broker connectivity and reconciliation:

```
python -m alpaca_agents.executor account
python -m alpaca_agents.executor reconcile
```

account reports selected broker fields. It does not verify settled cash, history coverage, or inventory consistency. reconcile returns detailed reasons and a snapshot valid for the rules engine only within 60 seconds. A successful check authorizes no order by itself.

Outside a session, MARKET_CLOSED or NOT_A_SESSION can be expected. Do not dismiss additional reasons simply because the market is closed. During an open session, success means reconciled: true and an empty reasons list.

Run the standalone scanner

```
python -m alpaca_agents.scanner --session 2026-09-17 `
--symbols SPY QQQ IWM --output runtime/shadow-scan.json
```

Replace the example date with the last completed exchange session. The scanner combines completed daily bars with current eligible option quotes; setting an old --session does not create an options historical backtest. Use it during an active market session when fresh quotes are available.

It cannot enable playbooks or place orders. Its report contains generated_at, sources, errors, proposals, shadow, and skipped. Snapshot errors make the command return a failure status even if other symbols were processed. Inspect individual symbols; do not reduce the result to one aggregate count.

Interpret the outcome correctly

- An error means the scanner could not establish valid inputs. No strategy conclusion follows.
- A valid skip means the data was processed but a signal, structure, price, liquidity, or portfolio condition failed.
- A shadow idea is a candidate that survived its scanner conditions while disabled for execution. It has not been approved for a trade.
- An enabled proposal is still subject to deterministic risk checks, journal validation, and broker submission gates.

Stocks data access does not imply access to current option quotes. Verify that your current Massive subscription supplies the exact snapshots, quotes, and freshness the code requires. Do not buy a plan solely because an old project note names a tier or price. Provider documentation is linked in Volume 3.

Keep a shadow log

For each session record: date, run time, bars session, symbols attempted, successful snapshots, errors, skips, ideas, and whether you understand each decision. Save interesting reports under dated filenames before the default report is overwritten. Rebuild the dashboard after a new scan.

Ten clean sessions is a useful project practice target, not an automatic promotion rule. Include sessions with no signals. If options data is unavailable, continue underlying research and local dashboard learning, but do not count those days as successful option-scanner validation.

7. Know exactly what can become an order

Signal and contract filters

The trend signal uses EMA20/EMA50 ordering and 20-session momentum, with pullback or prior-bar-break logic. It rejects chasing more than its configured distance and stops narrower than 0.5 times ATR14. The bounce signal looks for an oversold condition within a long-term uptrend. Breakout uses a tight range near highs and a volume-confirmed break.

Contract selection currently requires 30-45 calendar days to expiry, positive two-sided prices, open interest of at least 500, and a bid-ask width no greater than 10% of the midpoint. Long options target absolute delta near 0.40 within 0.35-0.45. Eligible long entries are costed at the ask. Quotes must pass the adapter's freshness and metadata checks.

These filters can make qualifying contracts rare. A strong underlying signal does not guarantee an affordable 30-45 DTE option. Do not widen limits merely to make the activity panel less empty.

Risk is total debit plus estimated fees

The entry rule is $\text{limit_debit} \times 100 \times \text{quantity} + \text{estimated_fees} \leq \100 . Exactly one contract, or one validated spread unit at the rules layer, is allowed; the executor currently rejects spreads. The scanner's \$0.65 per-contract fee estimate is a placeholder, not a current broker tariff or a verified round-trip cost.

Illustration: a \$0.90 quoted option with one contract and \$0.65 estimated fees uses \$90.65 of modeled entry risk. A \$1.00 premium plus those fees is \$100.65 and exceeds the cap. Do not confuse the option's per-share quote with its 100-share contract cost.

Shared account constraints

The rules allow at most two open plus pending entries, reject another entry after five attempts in a rolling hour, and require sufficient reconciled settled cash. The controller considers at most one new entry per cycle and prevents a new idea for an underlying already held or pending. Spendable cash is constrained by the local \$2,000 capital cap minus open basis as well as settlement checks.

The daily breaker uses cumulative realized losses and relevant fees, without letting wins erase losses. For illustration, $-\$25, +\30 , then $-\$20$ gives $+\$5$ net but $\$45$ gross losses before any additional fees. The loss gate has been crossed. It blocks new entries; it does not cap total losses on open positions at \$40.

Readiness requires all gates together

New paper orders require an approved playbook marker, the matching `--enable-playbook` launch flag, `ARMED_PAPER`, `--submit`, fresh reconciliation, eligible data/structure, and a successful journal authorization. A green dashboard component proves only that component. A successful backtest is not an approval marker; an approval marker is not an active launch flag.

8. Conduct a controlled paper pilot

Prepare a written decision first

Choose one executable playbook and state the symbol universe explicitly. Explain what evidence supports the test, what remains unknown, what would stop the pilot, and when it will be reviewed. The example below uses `trend_directional` and `QQQ` solely to demonstrate syntax; it is not a recommendation to trade `QQQ` or an assertion of an edge.

Stop an existing controller cleanly before changing launch configuration. Its approved/enabled playbook set is loaded at startup; do not rely on editing approval files mid-run to reconfigure it. The trading-control file is checked during operation and is the immediate control mechanism.

Approve and select the mode deliberately

These commands change local trading permissions. Use them only after completing your own review, with the controller stopped and the intended runtime directory confirmed.

```
@'
from pathlib import Path
Path('runtime/playbooks').mkdir(parents=True, exist_ok=True)
Path('runtime/playbooks/trend_directional.approved').write_text('APPROVED', encoding='utf-8')
'@ | python
@'
from pathlib import Path
Path('runtime/trading-control').write_text('ARMED_PAPER', encoding='utf-8')
'@ | python
```

Optional dry entry-cycle check

```
python -m alpaca_agents.controller --runtime runtime `
--symbols QQQ --enable-playbook trend_directional --dashboard
```

No `--submit` means no POST, but this is not purely read-only: an approved candidate can create a temporary entry reservation that expires. The current authorization TTL is 30 seconds. If there is no candidate, an empty entry list is expected. A dry run does not prove a future order will fill.

Launch the supervised paper session

```
python -m alpaca_agents.controller --runtime runtime `
--symbols QQQ --every 300 --dashboard `
--enable-playbook trend_directional --submit
```

This command can submit paper orders. Keep the terminal open, confirm initial reconciliation, and inspect every first-session order transition. To add browser chat to the same process, include `--serve`. Do not run a separate served controller on the same runtime.

The five-minute setting is a polling interval, not an exact wall-clock schedule. Work takes time, the loop can stop, and the computer can sleep. There is no built-in daily auto-start service. Verify the next day's launch instead of assuming yesterday's terminal still manages positions.

Verify outcomes rather than inferring them

A submitted response is not a fill. Check broker order status, imported fills, local inventory, and the next reconciled cycle. Do not click repeatedly or rerun a manual order command after an ambiguous response. Preserve the `client_order_id` and request trace for investigation.

9. Understand how exits really work

Priority after the entry session

Automatic exit evaluation uses this order: underlying stop, underlying rule such as an EMA20 close-through, premium stop, underlying target, then time stop. The first matching rule supplies the recorded reason. Multiple conditions can be true at once; a different recorded reason is not necessarily an error.

Underlying conditions use the last completed session's close, not a continuously observed intraday price touch. The premium check uses the broker option mark at evaluation time. This is a polling exit manager, not a resting broker stop order.

Entry-day behavior is narrower

On the session a position was opened, only the premium stop is evaluated. There is no same-session target or time exit from these automatic rules. The code permits this protective same-session exit only while fewer than three counted day trades have been used in its rolling five-session window.

Treat that as this version's implemented policy. It is not a complete statement of current broker eligibility or trading regulation. If the budget is exhausted, the code can hold the position despite a premium-stop condition; it does not guarantee a 50% maximum loss.

Automatic pricing versus manual flatten pricing

Automatic exits use a fresh, positive, two-sided option bid when available. Otherwise they can use a day-limit price at 95% of the broker mark, floored at \$0.01, provided the necessary inputs survived collection. A provider exception can instead leave exit inputs unavailable. Read the cycle record to distinguish a fallback price from a failed input stage.

The manual flatten command uses 95% of the broker mark; it does not retrieve the automatic manager's fresh bid. Both are limit orders. Neither pricing method guarantees execution. The application has no general cancel/replace or chase mechanism.

Time stops differ from backtest assumptions

The scanner's long-option plans use a 21-DTE time stop. The oversold_bounce plan also includes a ten-session limit. The trend_directional plan does not currently include the backtest's fifteen-session holding limit. This is an important implementation difference: replay holding-time assumptions are not automatically copied into execution plans.

An unfilled day order is not a closed position. An exit can be reevaluated in a later session with a fresh decision key, while outstanding or uncertain intents must still reconcile. Never assume a position is flat because an exit notification appeared.

Keep exit management operating when winding down

EXITS_ONLY prevents new entries while allowing exit handling. It still needs a running submission-enabled controller, valid inputs, and acceptable state. DISABLED prevents automatic exits as well as entries. Neither setting cancels an order already at the broker or reverses a fill.

10. Build an efficient daily and weekly routine

Before starting a session

Confirm the correct account/runtime pair, machine availability, current credentials, intended control mode, and launch flags. Review open positions and unresolved intents before looking at new ideas. Check that yesterday's controller stopped cleanly. If the market is open, run or inspect a fresh reconciliation and resolve every unexpected reason.

Pick the activity appropriate to your phase: cached research, standalone shadow scanning, disabled connectivity loop, or a reviewed paper pilot. Keep one concise session note that records what you intended to test. That makes unexpected activity easier to identify.

During a supervised pilot

Check the first cycle, then each initial entry/exit lifecycle. Watch timestamps and notifications, especially after a network interruption, laptop sleep, credential change, or subscription change. Once behavior is established, use scheduled check-ins appropriate to the system's polling frequency and the positions held; do not assume the dashboard is an alerting service.

If you need to step away, decide how positions will be managed before closing the terminal. Closing a browser tab does not close the controller. Closing the owning terminal generally stops the process and its automatic exit checks.

End-of-session review

```
Get-Content runtime\cycles.jsonl -Tail 3 |  
  ForEach-Object { $_ | ConvertFrom-Json } |  
    ConvertTo-Json -Depth 12  
Get-Content runtime\notifications.jsonl -Tail 15
```

Review reconciliation, exits, scan counts when present, entry decisions, and submission results. The actual entry records are under stages.entries.entries; scanner counts are under stages.scan. Not every stage exists in a halted or control-skipped cycle.

For each closed trade, document the original idea, accepted risk, actual fills, exit reason, costs, delays, and deviations. For each no-trade day, document whether it was a clean absence of candidates or an operational problem. Preserve rejected and failed cases; they are part of the denominator.

Weekly review

Separate process reliability from strategy observations. Count attempted sessions, clean sessions, data failures, unexplained reconciliation failures, order ambiguities, unfilled orders, and closed trades. Evaluate option results from actual fills and costs; do not substitute underlying backtest R.

Keep the playbook and limits unchanged during the observation window. If you make a change, record its date and begin a distinct cohort. A small-sample good week should not justify higher size, and a bad week should not trigger unrecorded parameter changes. Volume 2 includes reusable review templates.

11. Limits, evidence, and the next learning cycle

Known implementation gaps to remember

Multi-leg execution is unavailable even though the scanner and rules understand some spreads. Assignment and exercise activities are not a fully supported lifecycle. Worthless expiration has a normalization path, but the exact broker records still need to match supported shapes. Unknown activity or mismatched inventory blocks reconciliation.

The underlying replay excludes fills beyond stop/target, while the repository documents no equivalent current intraday underlying-entry recheck in execution. A daily signal can therefore be stale relative to the next session's movement. Treat this as a material research-to-execution gap when reviewing candidates and deciding whether the implementation is ready for a pilot.

Backtests use daily bars and simplified fills. They do not model option premiums, IV, theta, fees, or options liquidity. Automated target checks use completed closes, whereas replay can resolve targets from intraday daily-bar highs/lows. Time-stop assumptions also differ as described in chapter 9. Do not interpret replay performance as execution parity.

A useful decision record

Write: the question; code revision; bars and report filenames; test dates; playbook and universe; key statistics; operational defects; unresolved assumptions; decision; and next review date. Distinguish "reject this test configuration," "collect more evidence," and "eligible for a bounded paper pilot." Those are different conclusions.

The historical QQQ/SPY/IWM figures in the handover are useful audit context, not a current ranking or recommendation. Reproduce results from the intended data and current code. If a result changes, compare the inputs, source revision, and exclusions before explaining it as market behavior.

Your best next action

If setup is new, finish chapters 3-4. If you can already open the dashboard, reproduce one cached report and complete the shadow log in chapter 6. If you already have paper positions, prioritize fresh reconciliation, the exit-management state, and the incident reference over new strategy experiments.

Primary local sources: `controller.py`; `rules.py`; `scanner/scan.py`, `signals.py`, `contracts.py` and `__main__.py`; `executor/reconcile.py`, `orders.py`, `exits.py` and `__main__.py`; `dashboard.py`; `studio.py`; `llm_chat.py`; `backtest/__main__.py` and `replay.py`. All module paths are under `src/alpaca_agents/`. The companion reference includes external provider links and a source map.